

Theater Ticketing System

Software Requirements Specification

V6

02/12/2026

Group 7

Alvin Hoang, Brandon Tse, David Nguyen

Prepared for
CS 250 - Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Spring 2026

Revision History

Date	Description	Author	Comments
2/12/26	Version 1	David Nguyen	First Revision
2/12/26	Version 1	Alvin Hoang	First Revision
2/12/26	Version 1	Brandon Tse	First Revision
2/19/26	Version 2	David Nguyen	Second Revision
2/19/26	Version 2	Alvin Hoang	Second Revision
2/19/26	Version 2	Brandon Tse	Second Revision
2/19/26	Version 3	David Nguyen	Third Revision
3/5/26	Version 4	David Nguyen	Fourth Revision
3/5/26	Version 4	Alvin Hoang	Fourth Revision
3/5/26	Version 4	Brandon Tse	Fourth Revision
3/19/26	Version 5	David Nguyen	Fifth Revision
3/19/26	Version 5	Alvin Hoang	Fifth Revision
3/19/26	Version 5	Brandon Tse	Fifth Revision
4/9/26	Version 6	David Nguyen	Sixth Revision
4/9/26	Version 6	Alvin Hoang	Sixth Revision
4/9/26	Version 6	Brandon Tse	Sixth Revision

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	David Nguyen	Software Engineer	02/17/26
Brandon Tse	Brandon Tse	Software Engineer	02/17/26
Alvin Hoang	Alvin Hoang	Software Engineer	02/17/26
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History.....	2
Document Approval.....	2
1. Introduction.....	7
1.1 Purpose.....	7
1.2 Scope.....	7
1.3 Definitions, Acronyms, and Abbreviations.....	8
1.4 References.....	8
1.5 Overview.....	8
2. General Description.....	9
2.1 Product Perspective.....	9
2.3 User Characteristics.....	9
2.4 General Constraints.....	10
2.5 Assumptions and Dependencies.....	10
3. Specific Requirements.....	11
3.1 External Interface Requirements.....	11
3.1.1 User Interfaces.....	11
3.1.2 Hardware Interfaces.....	11
3.1.3 Software Interfaces.....	11
3.1.4 Communications Interfaces.....	11
3.2 Functional Requirements.....	11
3.2.1 Process Ticket Payment.....	11
3.2.1.1 Introduction.....	11
3.2.1.2 Inputs.....	12
3.2.1.3 Processing.....	12
3.2.1.4 Outputs.....	12
3.2.1.5 Error Handling.....	12
3.2.2 Payment Authorization Validation.....	12
3.2.2.1 Introduction.....	12
3.2.2.2 Inputs.....	12
3.2.2.3 Processing.....	13
3.2.2.4 Outputs.....	13
3.2.2.5 Error Handling.....	13
3.2.3.1 Seat Reservation.....	13
3.2.3.2 Inputs.....	13
3.2.3.3 Processing.....	13
3.2.3.4 Outputs.....	13
3.2.3.5 Error Handling.....	14
3.2.4 Ticket Purchase Restrictions.....	14
3.2.4.1 Processing.....	14

3.2.4.2 Error Handling.....	14
3.2.5 Bot Prevention.....	14
3.2.5.1 Processing.....	14
3.2.6 Discount Processing.....	14
3.2.6.1 Processing.....	14
3.2.7 Administrator Management.....	15
3.2.7.1 Processing.....	15
3.3 Use Cases.....	15
3.3.1 Complete Ticket Payment.....	15
3.3.2 Retry Payment.....	16
3.3.3 Reserve Seat.....	16
3.4 Classes / Objects.....	17
3.4.1 Payment.....	18
3.4.2 Transaction.....	18
3.4.3 Reservation.....	19
3.4.4 Movie.....	19
3.4.5 Showtime.....	20
3.5 Non-Functional Requirements.....	20
3.5.1 Performance.....	20
3.5.2 Reliability.....	21
3.5.3 Availability.....	21
3.5.4 Security.....	21
3.5.5 Maintainability.....	21
3.5.6 Portability.....	21
3.6 Inverse Requirements.....	21
3.7 Design Constraints.....	21
3.8 Logical Database Requirements.....	22
3.9 Other Requirements.....	22
4. Analysis Models.....	22
4.1 Sequence Diagrams.....	23
4.3 Data Flow Diagrams (DFD).....	23
4.2 State-Transition Diagrams (STD).....	24
5. Change Management Process.....	25
A. Appendices.....	25
A.1 Appendix 1.....	26
A.2 Appendix 2.....	26
6. Software Design Specification (Design Document).....	26
6.1 System Description (Developer Overview).....	26
6.2 Software Architecture Overview.....	26
6.2.4 Updated Architecture Diagram Explanation.....	29
6.2.5 Data Management Strategy.....	29

6.2.6 Design Decisions and Tradeoffs.....	29
6.3 UML Class Diagram (Design-Level).....	30
6.4 Class Specifications.....	30
6.5 Development Plan and Timeline.....	32
Task 1 - Project Setup and CI.....	32
Task 2 - Database Schema and ORM.....	32
Task 3 - Reservation and Seat Locking.....	33
Task 4 - Checkout and Payment Integration.....	33
Task 5 - Ticket Generation and Email.....	33
Task 6 - Administrative Features.....	34
Task 7 - Testing and System Hardening.....	34
6.5.2 Team Member Responsibilities.....	34
Alvin Hoang.....	34
Brandon Tse.....	34
David Nguyen.....	34
6.4.5 User.....	35
6.4.6 Movie.....	35
6.4.7 Theater.....	36
6.4.8 Showtime.....	36
6.4.9 Seat.....	36
6.4.10 Discount.....	37
6.4.11 PaymentGatewayAdapter.....	37
6.4.12 EmailServiceAdapter.....	38
6.4.13 CaptchaAdapter.....	38
6.4.14 PaymentDetails.....	38
6.4.15 GatewayResult.....	38
6.4.16 ReservationService (Design Notes).....	39
6.4.17 PaymentService.....	39
6.4.18 TicketService (Design Notes).....	39
6.5.3 Project Timeline (17 Week Schedule).....	40
Weeks 1-3.....	40
Weeks 4-6.....	40
Weeks 7-8.....	40
Weeks 9-11.....	40
Weeks 12-13.....	40
Weeks 14-15.....	40
Weeks 16-17.....	41
7. Verification and Validation Test Plan.....	41
7.1 Purpose and Scope.....	41
7.2 Design Specification Update.....	41
7.3 Test Strategy Overview.....	44

7.4 Test Environment and Data.....	44
7.5 Unit Test Set U-A - Payment and Policy Validation.....	44
7.6 Unit Test Set U-B - Reservation State and Seat Lock Logic.....	45
7.7 Functional / Integration Test Set F-A - Checkout Integration.....	45
7.8 Functional / Integration Test Set F-B - Admin Update Propagation.....	46
7.9 System Test Set S-A - Performance, Concurrency, and Abuse Handling.....	46
7.10 System Test Set S-B - Cross-Platform Acceptance.....	46
7.11 Requirements Traceability Matrix.....	47
7.12 Exit Criteria.....	47

1. Introduction

The Theater Ticketing System is a web-based movie ticketing platform that is designed to support both online customers and in-person theater employees. In addition, it will also maintain synchronized, real-time ticket availability across all theaters. This Software Requirements Specification defines the functional and non-functional requirements necessary to design, implement, test, and deploy the system successfully.

The system will serve 20 theaters located in the San Diego area and operate within the Pacific Time Zone. It will replace the current slow and error-prone ticketing system with a more reliable, scalable, and secure solution.

1.1 Purpose

The purpose of this SRS is to describe the functional and non-functional requirements of the Theater Ticketing System.

This document provides a detailed description of the system's capabilities, interfaces, constraints, and expectations. It is supposed to be a reference for developers, project managers, stakeholders, and testers involved in the design, implementation, and maintenance of the system.

1.2 Scope

The Theater Ticketing System is a web-based application designed to allow customers to look at movies, select showtimes, reserve seats, and purchase tickets online.

The system will:

- Provide real-time seat availability
- Enable secure online payments
- Generate barcodes and QR codes for each respective ticket
- Send a confirmation email
- Provide tools for the owners so that they're able to manage each movie, its pricing, and schedules.
- Support ticket purchases up to two weeks in advance
- Support assigned seating for deluxe theaters and general admission for regular theaters
- Allow discount pricing for students, military personnel, and seniors

The system will not:

- Process refunds online
- Sell concessions
- Provide streaming services

The goal of the system is to streamline ticket purchasing, reduce wait times for purchasing tickets in person, and to also improve ease of changing movie parameters such as schedules and pricing.

1.3 Definitions, Acronyms, and Abbreviations

This subsection should provide the definitions of all terms, acronyms, and abbreviations required to properly interpret the SRS. This information may be provided by reference to one or more appendixes in the SRS or by reference to other documents.

This document follows standard IEEE SRS formatting conventions:

- “Shall” indicates a mandatory requirement.
- “Should” indicates a recommended requirement.
- “May” indicates an optional feature.
- Numbered sections are hierarchical and organized by feature category.
- Functional requirements are grouped under Section 3.2 and follow a structured format including inputs, processing, outputs, and error handling.

1.4 References

PCI Security Standards Council. (2024). *Payment Card Industry Data Security Standard: Requirements and Testing Procedures (Version 4.0.1)*. PCI Security Standards Council. https://docs-prv.pcisecuritystandards.org/PCI%20DSS/Standard/PCI-DSS-v4_0_1.pdf

1.5 Overview

The remaining sections of this SRS provide a general description, including characteristics of the users of this project, the product's hardware, and the functional and data requirements of the product. The general description of the project is discussed in section 2 of this document. Section 3 gives the functional requirements, data requirements and constraints and assumptions made while designing the Ticketing System. It also gives the user viewpoint of the product. Section 3 also gives the specific requirements of the product. Section 3 also discusses the external interface requirements and gives detailed description of functional requirements. Section 4 is for supporting information.

The remainder of this SRS is organized as follows:

- Section 2 provides a general description of the system, including user characteristics, constraints, and assumptions.
- Section 3 describes the specific functional and non-functional requirements of the system.
- Section 4 presents analysis models such as sequence diagrams, data flow diagrams, and state-transition diagrams.
- Section 5 describes the change management process for maintaining and updating system requirements.

- The appendices contain supporting documentation and additional reference material.

2. General Description

This section of the SRS should describe the general factors that affect 'the product and its requirements'. It should be made clear that this section does not state specific requirements; it only makes those requirements easier to understand.

2.1 Product Perspective

The Theater Ticketing System is a web based system that uses other services such as payment systems as well as email services.

It will interact with: A database for data storage, a payment gateway so that people can pay with online services, an email system so that the ticketing system can send email confirmation, and other hardware such as ticket printers and barcode scanners.

2.2 Product Functions

The Major functions of the System will be:

- Account registration + login
- Movie Searching
- Time selection for each movie
- Seat selection with a chart of the theater
- Ticket reservation
- Cart for tickets
- Online payment processing
- Generation of tickets with ID and QR code
- Email confirmation
- Allow Owner to manage movies, pricing, and schedule

2.3 User Characteristics

Customers:

- Audience that will purchase tickets
- Basic knowledge of the internet
- Access to the ticketing system with either a web browser on a mobile or desktop device

Owner:

- Theater management or other staff
- Manage movies, schedules, and pricing
- Need secure login information
- Moderate knowledge of the internet

System Admin:

- Responsible for maintenance of system

- Manage database
- Advanced knowledge of the internet

2.4 General Constraints

- The system will use HTTPS for communications
- Payment processing will comply with PCI-DSS
- It will need a payment system such as Stripe
- The database must be relational
- It will comply with privacy regulations
- Passwords will be stored using secure methods
- The system shall be compatible with major web browsers (Chrome, Edge, Safari, Firefox).
- The system shall operate within standard internet bandwidth conditions.
- The system shall not require specialized hardware for customers.
- Administrative users shall require secure authentication credentials.

2.5 Assumptions and Dependencies

Assumptions:

- The theater will have reliable internet connection so that people can buy tickets online
- Users have access to a web browser and will be able to connect to the internet
- The payment service will be operational
- The email provider will be able to reliably email ticket information to customers
- Theater staff will have the ability to use the administration tools
- The seating layout will be predetermined and will be consistent
- The information that the customer provides about their payment information will be correct
- The system will be hosted in an environment that has adequate capacity
- It is assumed that users have access to reliable internet connectivity.
- It is assumed that third-party services (payment gateway and email provider) remain operational.
- It is assumed that the hosting environment supports scalable concurrent user traffic.

Dependencies:

- Payment system: a third party payment system to handle transactions securely
- Email provider: external email service to send ticket confirmation
- Database: needs a database in order to store movie information, schedules, seat availability, user data, and transaction history
- Barcode/Qr code library: needs a system to generate barcodes and QR codes
- Hardware: ticket printers and/or barcode scanners for digital tickets.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- The system shall provide a secure web-based payment interface accessible via modern browsers (Chrome, Edge, Safari, Firefox).
- The interface shall display:
 - Reservation ID
 - Total payment amount
 - Payment method options (credit/debit card, supported digital wallets)
- The system shall display a confirmation message after payment approval or a failure message if payment is declined.

3.1.2 Hardware Interfaces

- The system shall operate on standard user devices (desktop, tablet, mobile).
- No specialized hardware shall be required for payment processing.

3.1.3 Software Interfaces

- The system shall integrate with a third-party payment gateway (e.g., Stripe, PayPal).
- The system shall interface with the reservation database to verify active reservations before processing payment.
- The system shall integrate with an email service to send payment confirmations.

3.1.4 Communications Interfaces

- All communication between the user and system shall use HTTPS.
- Payment transactions shall use encrypted API communication with the payment gateway.

3.2 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

3.2.1 Process Ticket Payment

3.2.1.1 Introduction

This function processes customer payments for reserved movie tickets.

3.2.1.2 Inputs

- Reservation ID
- Total payment amount
- Card number or digital wallet credentials
- Expiration date
- CVV
- Billing address

3.2.1.3 Processing

- Validate required input fields.
- Verify reservation ID is valid and active.
- Confirm payment amount matches reservation total.
- Submit transaction request to payment gateway.
- Receive transaction approval or denial.
- Store transaction record in database.

3.2.1.4 Outputs

- Payment confirmation message (success or failure).
- Transaction ID (if successful).
- Electronic receipt sent via email.

3.2.1.5 Error Handling

- If required fields are missing, display error message.
- If reservation ID is invalid, cancel transaction and notify user.
- If payment is declined, notify user and allow retry.
- If gateway communication fails, log error and prompt retry.

3.2.2 Payment Authorization Validation

3.2.2.1 Introduction

This function validates user payment information and authorizes the transaction before final processing.

3.2.2.2 Inputs

- Card number or digital wallet credentials
- Expiration date
- CVV
- Billing address

- Reservation ID
- Total amount

3.2.2.3 Processing

- Validate input format (card length, expiration date format, required fields).
- Verify billing information is complete.
- Confirm reservation ID is valid and active.
- Send authorization request to payment gateway.
- Receive authorization approval or denial.

3.2.2.4 Outputs

- Authorization approval or denial message.
- Authorization code (if approved).

3.2.2.5 Error Handling

- Display error if required fields are missing.
- Display error if card format is invalid.
- Display message if authorization is declined.
- Log failed authorization attempts for security review.

3.2.3.1 Seat Reservation

This function allows customers to temporarily reserve selected seats for a specific showtime.

3.2.3.2 Inputs

- Movie ID
- Showtime ID
- Selected seat numbers

3.2.3.3 Processing

- Verify selected seats are available.
- Lock seats for a maximum of 10 minutes.
- Generate Reservation ID.
- Update database to reflect temporary hold.

3.2.3.4 Outputs

- Reservation ID
- Confirmation message

3.2.3.5 Error Handling

- If seats are unavailable, display error message.
- If session expires, release seats automatically.

3.2.4 Ticket Purchase Restrictions

This function will restrict customers from abusing the ticket system and balance the system to make it fair.

3.2.4.1 Processing

- Customers shall not purchase more than 20 tickets per transaction.
- Tickets shall only be available 14 days prior to showtime.
- Ticket purchases shall be disabled 10 minutes after showtime begins.

3.2.4.2 Error Handling

- Display a restriction message if limits are exceeded.

3.2.5 Bot Prevention

This function will prevent spam from occurring in the system and abuse of bots to get tickets.

3.2.5.1 Processing

- The system shall implement CAPTCHA verification at checkout.
- The system shall detect rapid repeated purchase attempts.
- Suspicious IP addresses shall be temporarily blocked.

3.2.6 Discount Processing

3.2.6.1 Processing

- The system shall verify discount eligibility category.
- The system shall apply discount percentage automatically.
- The system shall log discount usage.

3.2.7 Administrator Management

3.2.7.1 Processing

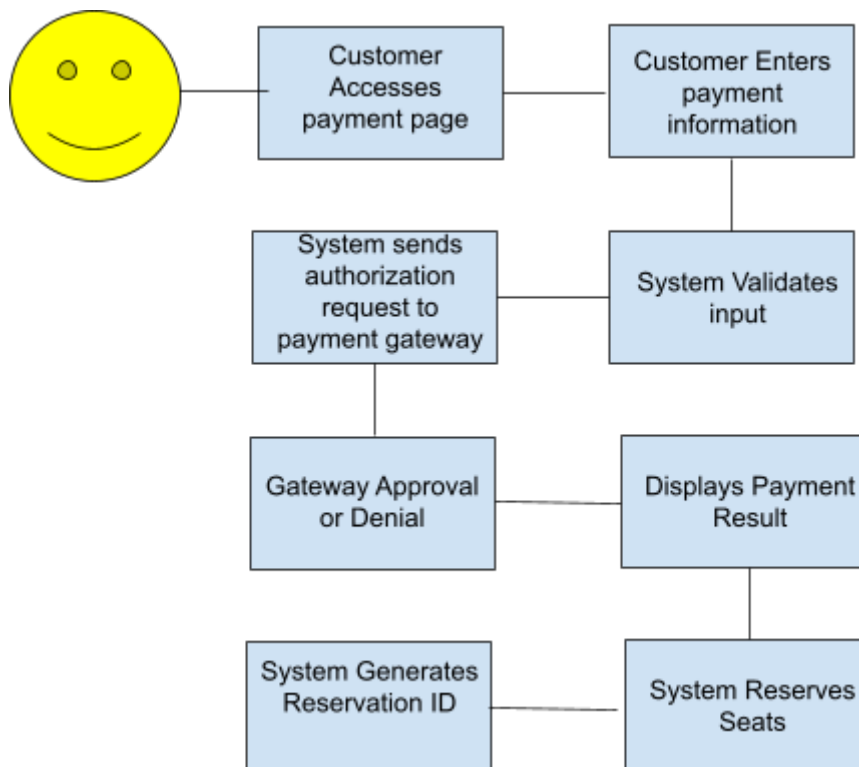
- Add, update, or remove movies.
- Modify ticket pricing.
- Configure showtimes.
- Update seating layout.

3.3 Use Cases

3.3.1 Complete Ticket Payment

Actor: Customer

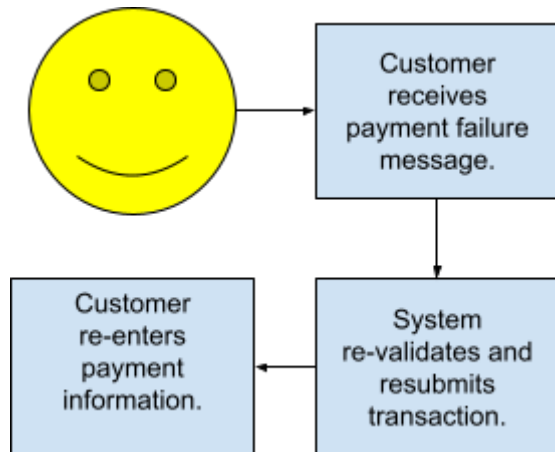
1. Customer accesses payment page.
2. Customer enters payment information.
3. System validates input.
4. System sends authorization request to payment gateway.
5. Gateway returns approval or denial.
6. System displays payment result.
7. System stores transaction record and sends receipt (if approved).



3.3.2 Retry Payment

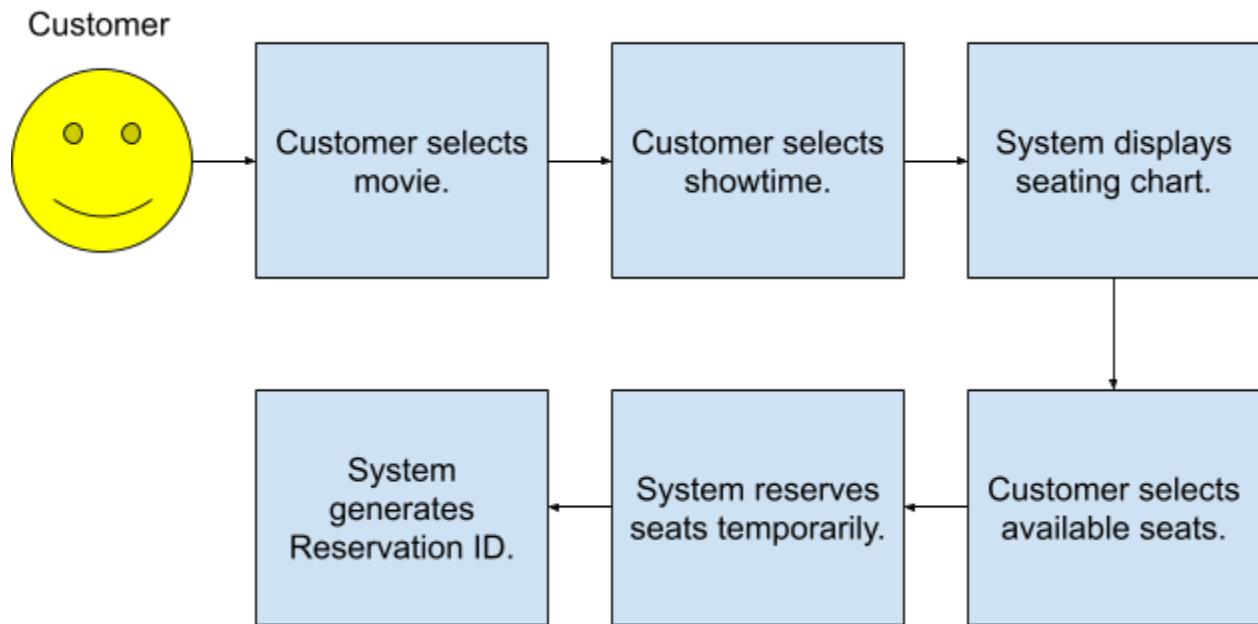
Actor: Customer

1. Customer receives payment failure message.
2. Customer re-enters payment information.
3. System re-validates and resubmits transaction.



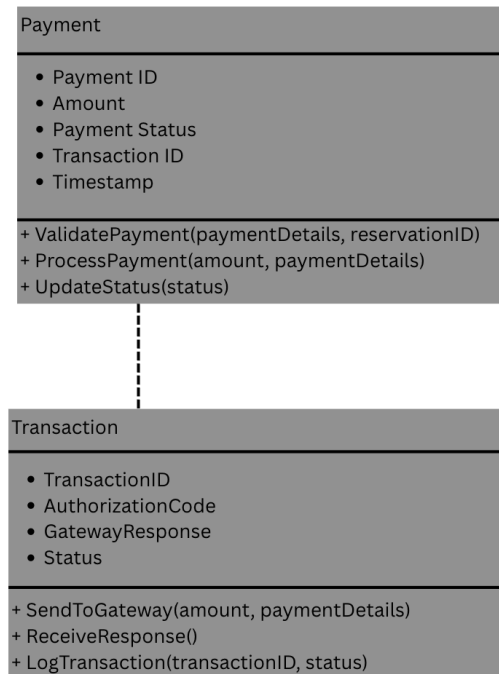
3.3.3 Reserve Seat

1. Customer selects movie.
2. Customer selects showtime.
3. System displays seating chart.
4. Customer selects available seats.
5. System reserves seats temporarily
6. System generates Reservation ID.



3.4 Classes / Objects

UML Diagram for Payment and Transaction Processes



3.4.1 Payment

3.4.1.1 Attributes

- PaymentID
- ReservationID
- Amount
- PaymentStatus
- TransactionID
- Timestamp

3.4.1.2 Functions

- ValidatePayment()
- ProcessPayment()
- UpdateStatus()

(References: 3.2.1, 3.2.2)

3.4.2 Transaction

3.4.2.1 Attributes

- TransactionID

- AuthorizationCode
- GatewayResponse
- Status

3.4.2.2 Functions

- SendToGateway()
- ReceiveResponse()
- LogTransaction()

(References: 3.2.1, 3.2.2)

3.4.3 Reservation

3.4.3.1 Attributes

- ReservationID
- ShowtimeID
- SeatNumbers
- Status (Pending, Confirmed, Expired)
- Timestamp

3.4.3.2 Functions

- LockSeats()
- ReleaseSeats()
- ConfirmReservation()

3.4.3.3 States

- Pending
- Confirmed
- Expired

3.4.3.4 Relationships

- Associated with Payment
- Associated with Showtime

3.4.4 Movie

3.4.4.1 Attributes

- MovieID

- Title
- Description
- Duration
- Rating

3.4.4.2 Functions

- AddMovie()
- UpdateMovie()
- RemoveMovie()

3.4.5 Showtime

3.4.5.1 Attributes

- ShowtimeID
- MovieID
- TheaterID
- DateTime
- AvailableSeats

3.4.5.2 Functions

- UpdateAvailability()
- CloseShowtime()

3.5 Non-Functional Requirements

Non-functional requirements may exist for the following attributes. Often these requirements must be achieved at a system-wide level rather than at a unit level. State the requirements in the following sections in measurable terms (e.g., 95% of transaction shall be processed in less than a second, system downtime may not exceed 1 minute per day, > 30 day MTBF value, etc).

3.5.1 Performance

- 95% of payment transactions shall be completed within 3 seconds.
- Authorization responses shall be processed within 2 seconds under normal operating conditions.
- The system shall support at least 1000 concurrent users without performance degradation.
- Seat reservation conflicts shall not exceed 2% under peak load.

3.5.2 Reliability

- The system shall maintain a 99% successful transaction processing rate, excluding user-declined payments.
- Failed transactions shall be logged for audit purposes.

3.5.3 Availability

- The payment processing module shall maintain 99% uptime per month.

3.5.4 Security

- All payment data shall be encrypted in transit using TLS 1.2 or higher.
- The system shall comply with PCI-DSS standards.
- The system shall not store full credit card numbers.
- The system shall not store CVV codes after transaction completion.
- The system shall implement CAPTCHA validation during checkout.
- Repeated failed transactions from the same IP address shall trigger temporary account or IP restriction.

3.5.5 Maintainability

- The payment gateway integration shall be modular to allow replacement without affecting other system components.
- System logs shall be maintained to assist troubleshooting and auditing.

3.5.6 Portability

- The payment processing module shall function on major web browsers.
- The system shall be accessible on Windows, macOS, Android, and iOS devices via browser.

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constraints imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

- *Security constraints*
 - *Payment must be handled by a third-party gateway.*
- *Time constraints*

- *Seat reservations will expire after 10 minutes if purchase isn't finished*
- *Tickets become available 14 days before showtime*
- *Platform Constraints*
 - *Must run as a web-based system accessible by common web browsers*
- *Anti-bot-constraints*
 - *Must include CAPTCHA*

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

A relational database will be used to store users, movies, showtimes, seats, reservations, payments, transactions, discounts, and tickets

- *Identifiers*
 - *Use UUID for primary values such as Movie, showtime, reservation, payment, transaction, ticket*
- *Money*
 - *Store currency values as amountCents: BIGINT*
- *Seat Representation*
 - *Seat numbers should be stored as strings such as A10 or combined as row:CHAR + seat: INT*
- *To prevent seats being double booked*
 - *Unique constraints should be used such as (showtime_id, seat_number for confirmed seats*
 - *Reservation expiration will automatically release seats*
- *Retention*
 - *Transaction records are retained for accounting and potential manual refund processing outside the system*
 - *Store only the payment gateway and not the actual card information*

3.8 Logical Database Requirements

We use 1 relational database and 1 temporary cache for seat locking.

- The relational database stores:
 - Users
 - Movies
 - Showtimes
 - Seats
 - Reservations
 - Payments
 - Transactions
 - Tickets
 - Discounts
- The Cache stores:

- Temporary seat locks
- Expiration timers

When a user selects seats, the system first locks seats in the cache. Once payment is successful, the reservation and payment are permanently stored in the relational database and the seats are marked as sold.

The system uses a hybrid data management strategy consisting of a relational database and a temporary in-memory cache. The relational database serves as the system of record and stores all persistent data including users, movies, showtimes, seats, reservations, payments, transactions, tickets, and discounts.

A temporary cache (e.g., Redis) is used to store short-lived seat locks during the reservation process. This allows the system to handle high concurrency and prevent double booking without overloading the main database.

The separation ensures that critical business data remains consistent and reliable while temporary operational data can be handled efficiently. Sensitive data such as payment information is not stored directly, and all transactions are handled through a secure third-party payment gateway.

3.8.2 Alternatives and Tradeoffs

One alternative design was to use only a relational database for all operations, including seat locking. While simpler, this approach would increase database load and reduce performance during peak usage.

Another alternative was to use a NoSQL database for all data storage. While this would improve scalability and flexibility, it would make it harder to enforce relationships and constraints between entities such as reservations, payments, and tickets.

The chosen approach uses a relational database for persistent data and a temporary cache for seat locking. This balances performance and data integrity. The tradeoff is increased architectural complexity, but this is justified by improved concurrency handling and system reliability.

3.9 Other Requirements

Catchall section for any additional requirements.

- *Email confirmation*
 - *After a successful purchase, system will send a confirmation email and receipt*
- *Usability*
 - *Seat selection UI must show real-time availability as well as showing which seats are already taken clearly*
- *Logging/monitoring*

- *Log checkout failures, suspicious activity, payment gateway erros and reservation expirations.*

4. Analysis Models

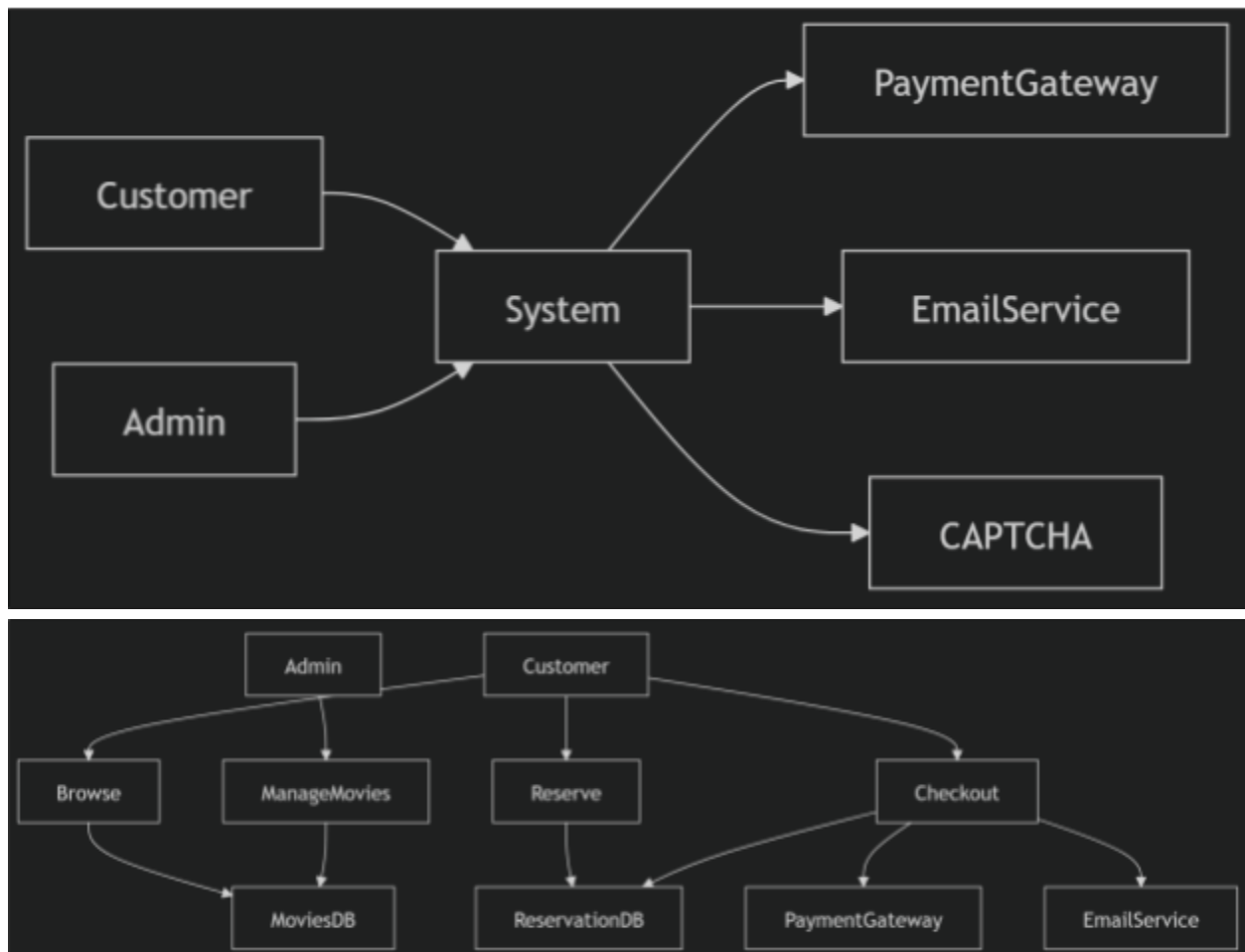
List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.

4.1 Sequence Diagrams

- Figure 1: Seat Reservation

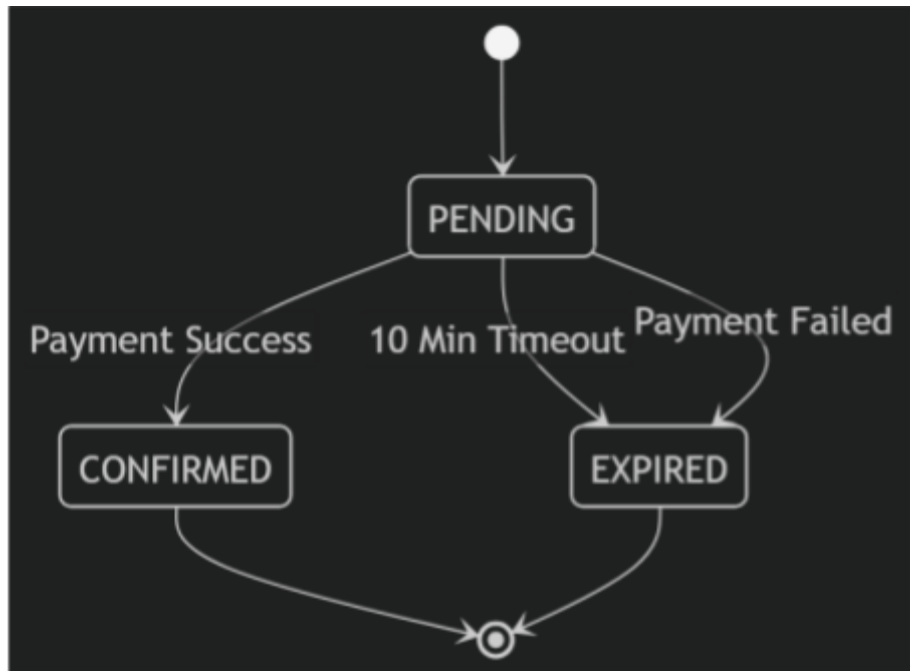


4.3 Data Flow Diagrams (DFD)

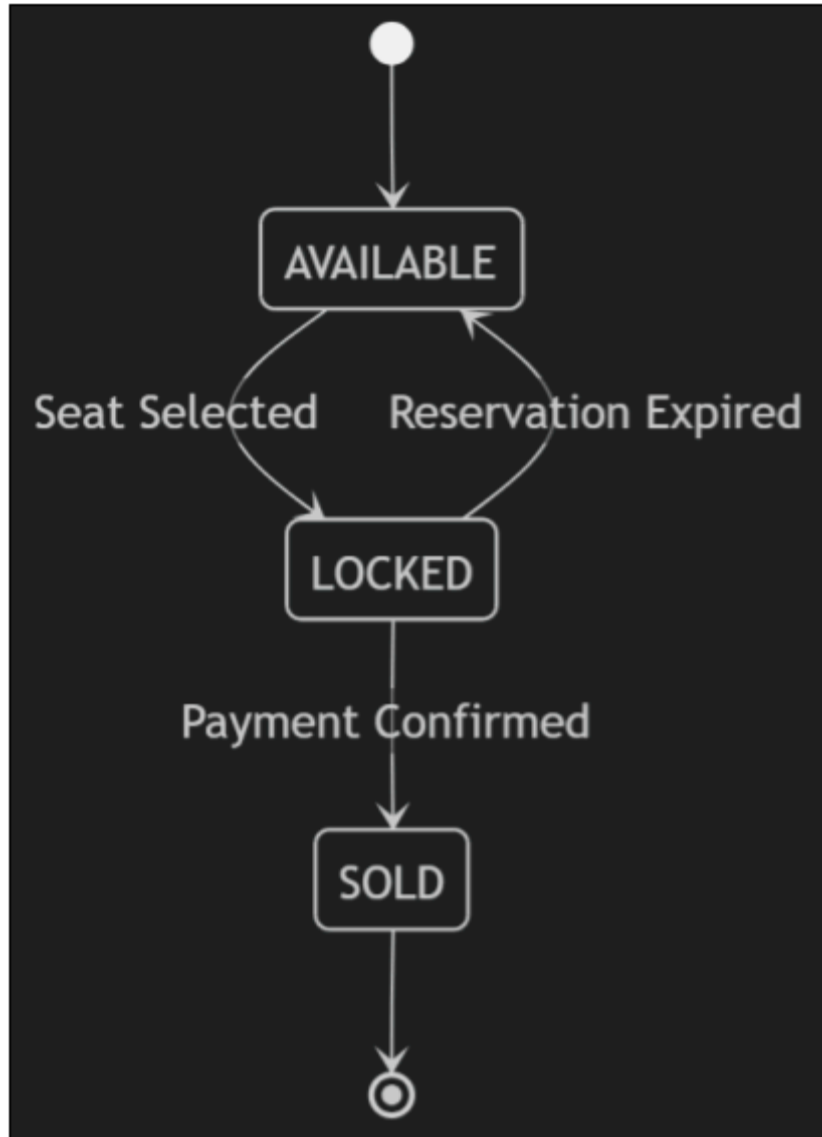


4.2 State-Transition Diagrams (STD)

Figure Reservation Diagram



Seat Diagram



5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2

6. Software Design Specification (Design Document)

6.1 System Description (Developer Overview)

The Theater Ticketing System is a web-based application supporting customers and theater staff for browsing movies, selecting showtimes, reserving seats, purchasing tickets, and receiving electronic tickets (QR/barcode). Administrative users manage movies, showtimes, pricing, and seating layouts. The system integrates with third-party services for payment processing, email delivery, and bot protection (CAPTCHA).

6.2 Software Architecture Overview

6.2.1 Architectural Style

The system uses a **3-tier architecture**:

1. **Presentation Layer (Web UI)**: Customer/admin web pages for browsing, reservation, checkout, and administration.
2. **Application Layer (REST API)**: Business logic services for reservations, payments, ticket generation, and admin management.
3. **Data Layer (Relational DB)**: Stores movies, showtimes, seats, reservations, payments, tickets, and audit logs.

External dependencies are accessed via service adapters (Payment Gateway, Email Service, CAPTCHA).

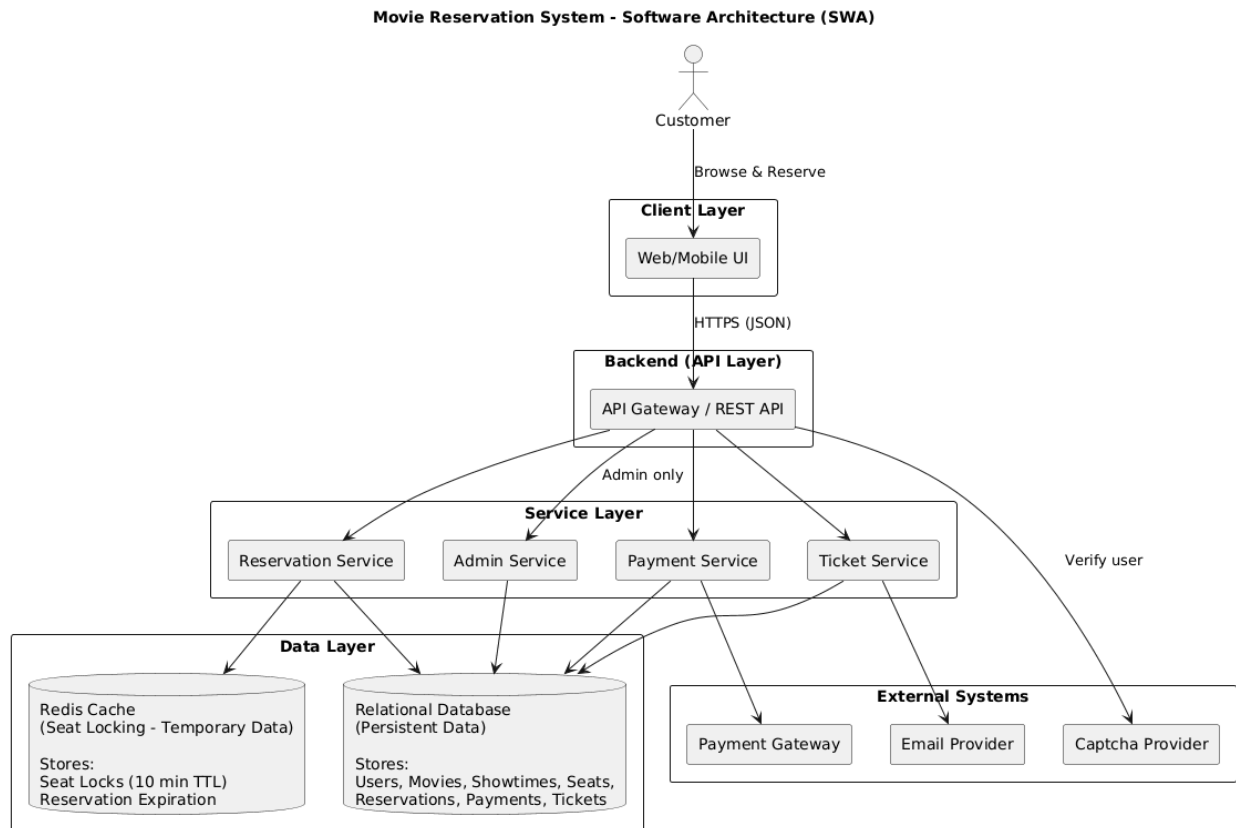
6.2.2 Major Components and Responsibilities

- **Web UI**
 - Customer: browse, select seats, checkout
 - Admin: manage movies/showtimes/pricing/seating layouts
- **API Gateway / Controllers**
 - Validates requests, invokes services, returns responses

- **Reservation Service**
 - Seat locking, reservation creation/expiration, conflict checks
- **Payment Service**
 - Payment authorization + charge, transaction recording, receipts
- **Ticket Service**
 - Generates ticket records and QR/barcode payloads
- **Admin Service**
 - CRUD for Movie/Showtime/Pricing/Layout
- **Database**
 - Persist all entities with integrity constraints
- **External Services**
 - Payment Gateway (Stripe/PayPal)
 - Email Provider
 - CAPTCHA Provider

6.2.3 Architecture Diagram (structure)

Theater Ticketing System Architecture Diagram



This software architecture diagram shows the overall structure of the movie reservation system and how all the main parts connect to each other. A customer uses the website or mobile app (the UI) to browse movies, pick a showtime, select seats, and check out. The UI sends requests over the internet (using HTTPS) to the backend API, which acts as the main entry point to the system. The API receives requests, checks that they are valid and that the user has permission, and then sends them to the correct internal service.

Behind the API are several services that each handle a specific responsibility. The Reservation Service handles seat selection and booking. The Payment Service processes payments and talks to an external payment provider to charge the customer. The Ticket Service creates tickets after payment is successful and sends them to the customer by email. The Admin Service allows staff to add movies, create showtimes, and manage theater information. Each service focuses on one job to keep the system organized and easier to maintain.

All of these services store and read information from the main database, which keeps records of users, movies, showtimes, seats, reservations, payments, and tickets. The system may also use a temporary locking system to hold seats for a short time so that two people cannot book the same seat at once. In addition, the system connects to outside services: a payment gateway to process credit cards, an email provider to send tickets and receipts, and a captcha service to prevent bots. Together, these parts show the complete structure of the system and how everything works together to support online movie reservations.

6.2.4 Updated Architecture Diagram Explanation

The architecture diagram has been updated from the previous design to better reflect the system's data management strategy. The updated version clearly includes the primary relational database, which stores core system data such as users, movies, showtimes, seats, reservations, payments, and tickets.

In addition, the design highlights how all backend services interact with the database to maintain consistency across reservation, payment, and ticketing processes. A temporary seat locking mechanism is also represented, which may use a caching layer to handle short-lived reservations and prevent double booking during checkout.

The updated diagram improves clarity by showing the separation between the presentation layer, application services, and data layer, while also emphasizing how external services such as the payment gateway, email service, and CAPTCHA integrate with the system.

Overall, these updates provide a clearer representation of how data is stored, accessed, and managed throughout the system.

6.2.5 Data Management Strategy

Database Choice:

The system uses a relational database (SQL) as the primary data store. This is because the system contains structured data with strong relationships between entities such as users, movies, showtimes, seats, reservations, payments, and tickets. A relational model ensures data integrity and reliable transaction handling.

Data Organization:

Data is logically separated by function:

- User Data: accounts, authentication, roles
- Movie/Theater Data: movies, theaters, showtimes, pricing
- Reservation Data: seat selection, reservation status, expiration
- Payment Data: payments, transactions, gateway responses
- Ticket Data: issued tickets and QR/barcode information

This separation improves maintainability, clarity, and query performance.

Seat Locking Strategy:

To prevent double booking, the system uses:

- database constraints (unique seat per showtime)
- temporary seat locking during checkout

Seats transition through states:

- Available → Locked → Sold

Locks automatically expire if payment is not completed.

Security and Sensitive Data:

- No full credit card data is stored
- Payment is handled through a third-party gateway
- Only non-sensitive data (e.g., transaction IDs) is stored
- All communication uses HTTPS

Data Retention and Integrity:

- Transactions and logs are stored for auditing
- Reservations automatically expire if not completed
- UUIDs are used as primary keys for uniqueness
- Monetary values are stored in cents to avoid rounding errors

6.2.6 Design Decisions and Tradeoffs

Relational Database vs NoSQL:

A relational database was chosen because the system requires strong consistency and clear relationships between entities.

- **Advantage:** data integrity and reliable transactions
- **Tradeoff:** less flexibility and scalability compared to NoSQL

Single Database vs Multiple Databases:

The system uses a single centralized database.

- **Advantage:** simpler design and easier transaction management
- **Tradeoff:** less scalability compared to distributed databases

Third-Party Payment Gateway:

A third-party payment service is used for processing transactions.

- **Advantage:** improved security and PCI compliance
- **Tradeoff:** dependency on external services and possible latency

Seat Locking with Cache:

A cache (e.g., Redis) may be used for temporary seat locking.

- **Advantage:** faster performance and better handling of concurrent users
- **Tradeoff:** added system complexity and synchronization requirements

- reservationId: UUID
- userId: UUID
- showtimeId: UUID
- status: ReservationStatus
- expiresAt: Instant

Operations:

- lockSeats(seatLabels: List<String>, holdMinutes: Integer): void
- releaseSeats(): void
- confirm(): void

6.4.2 Payment / Transaction

Purpose: Processes and records checkout payments.

Payment Attributes: paymentId: UUID, reservationId: UUID, amountCents: Long, status: PaymentStatus, createdAt: Instant

Payment Operations:

- validate(details: PaymentDetails): boolean
- process(details: PaymentDetails): Transaction
- updateStatus(newStatus: PaymentStatus): void

Transaction Attributes: transactionId: UUID, paymentId: UUID, gatewayTransactionId: String, authorizationCode: String, gatewayResponse: String, status: TransactionStatus

Transaction Operations:

- sendToGateway(amountCents: Long, details: PaymentDetails): GatewayResult
- log(): void

6.4.3 Ticket

Purpose: Represents issued entry tickets tied to a reservation.

Attributes: ticketId: UUID, reservationId: UUID, qrCodePayload: String, issuedAt: Instant

Operations:

- generateQrPayload(): String

6.4.4 AdminService

Purpose: Admin management of movies/showtimes/pricing.

Operations:

- addMovie(movie: Movie): UUID
- updateMovie(movieId: UUID, movie: Movie): void
- removeMovie(movieId: UUID): void
- createShowtime(showtime: Showtime): UUID
- updateShowtime(showtimeId: UUID, showtime: Showtime): void

6.5 Development Plan and Timeline

6.5.1 Task Partitioning (Work Packages)

Task 1 - Project Setup and CI

- Repository setup
- Branching strategy
- Linting configuration
- Basic CI pipeline setup

Task 2 - Database Schema and ORM

Tables to be created:

- Users
- Movies
- Theaters
- Showtimes
- Seats

- Reservations
- Payments
- Transactions
- Tickets
- Discounts

Constraints:

- Seat uniqueness per showtime
- Reservation expiration handling

Task 3 - Reservation and Seat Locking

- Seat availability checks
- 10-minute seat hold timer
- Conflict handling for simultaneous reservations

Task 4 - Checkout and Payment Integration

- Payment authorization and processing
- Transaction recording
- Error handling and retry logic
- CAPTCHA verification during checkout

Task 5 - Ticket Generation and Email

- Ticket record creation
- QR code payload generation
- Email confirmation and digital receipt

Task 6 - Administrative Features

- Manage movies, showtimes, pricing, and theater layouts
- Role-based access control for administrative users

Task 7 - Testing and System Hardening

- Unit testing for services
- Integration testing for reservation and payment flows
- Security checks and system logging

6.5.2 Team Member Responsibilities

Alvin Hoang

- Payment integration development
- Implementation of PaymentService and GatewayAdapter
- Payment and transaction database logic
- Security checks related to payment processing

Brandon Tse

- Reservation and seat locking APIs
- Customer interface for seat selection and checkout
- CAPTCHA integration for checkout verification

David Nguyen

- Database schema design and constraints
- Ticket generation and email integration
- Backend administrative services and API endpoints

Each team member will contribute multiple commits to the repository within their assigned areas.

6.4.5 User

Purpose: Represents a user account for system access and role-based permissions (Customer, Owner, Admin).

Attributes:

- **userId:** UUID -Unique identifier for the user
- **email:** String -Login identifier and contact email
- **passwordHash:** String -Secure hashed password (never store raw passwords)
- **role:** UserRole -Access level (CUSTOMER, OWNER, ADMIN)
- **createdAt:** Instant -Account creation timestamp

Operations:

- *(Authentication is handled in the API/controller layer; the User entity stores identity and role data.)*

6.4.6 Movie

Purpose: Stores metadata for a movie available for ticket purchase.

Attributes:

- **movieId:** UUID -Unique movie identifier
- **title:** String -Movie name
- **description:** String -Summary/description text
- **durationMinutes:** Integer -Runtime in minutes
- **rating:** String -Content rating (e.g., PG-13)
- **isActive:** Boolean- Indicates if movie is currently listed

Operations:

- *(CRUD operations are performed via AdminService using addMovie, updateMovie, removeMovie.)*

6.4.7 Theater

Purpose: Represents a physical theater location within the system (San Diego area).

Attributes:

- theaterId: UUID -Unique theater identifier
- name: String -Theater name
- location: String -City/address description
- timeZone: String -Time zone identifier (e.g., America/Los_Angeles)

Operations:

- *(Primarily data storage; configuration managed by Admin tools.)*

6.4.8 Showtime

Purpose: Represents a scheduled screening of a movie at a specific theater and time.

Attributes:

- showtimeId: UUID -Unique showtime identifier
- movieId: UUID -Linked movie
- theaterId: UUID -Linked theater
- startTime: Instant -Start time of showing
- isClosed: Boolean -If showtime is closed for purchase
- basePriceCents: Long -Base ticket price in cents

Operations:

- closeShowtime(): void -Disables ticket purchases for this showtime
- updateBasePrice(newPriceCents: Long): void -Updates base price for pricing changes

6.4.9 Seat

Purpose: Represents a seat inside a theater. Seat availability depends on showtime and reservation status.

Attributes:

- seatId: UUID -Unique seat identifier
- theaterId: UUID -Theater to which the seat belongs
- label: String -Seat label (e.g., A10)
- seatType: SeatType -STANDARD or DELUXE

Operations:

- *(Seat locking/sale is controlled through ReservationService and reservation state transitions.)*

6.4.10 Discount

Purpose: Stores and applies discount policies (student, military, senior).

Attributes:

- discountId: UUID - Unique discount identifier
- code: String - Discount code or category label
- type: DiscountType - STUDENT, MILITARY, SENIOR
- percentOff: Integer - Percentage discount applied (0-100)
- isActive: Boolean - Whether the discount is enabled

Operations:

- apply(amountCents: Long): Long -Returns the discounted amount in cents

6.4.11 PaymentGatewayAdapter

Purpose: Encapsulates communication with a third-party payment provider (Stripe/PayPal) so the system can swap gateways without rewriting business logic.

Operations:

- authorize(amountCents: Long, details: PaymentDetails): GatewayResult -Requests authorization for payment

- `capture(gatewayTransactionId: String): GatewayResult` -Captures/settles an authorized payment

6.4.12 EmailServiceAdapter

Purpose: Sends receipts and ticket confirmations after successful purchase.

Operations:

- `sendEmail(to: String, subject: String, body: String): void`

6.4.13 CaptchaAdapter

Purpose: Validates CAPTCHA tokens during checkout to reduce bot abuse.

Operations:

- `verify(token: String, ip: String): boolean`

6.4.14 PaymentDetails

Purpose: Standardizes payment input data used by payment processing functions without storing sensitive card data.

Attributes:

- `method: PaymentMethod` -CARD or WALLET
- `cardLast4: String` -Last 4 digits for display/logging (no full card storage)
- `billingZip: String` -Billing ZIP validation support
- `walletToken: String` -Token for digital wallet payment (if used)

Operations:

- *(Data transfer object; used as input parameter.)*

6.4.15 GatewayResult

Purpose: Standardizes payment gateway responses for application logic and logging.

Attributes:

- `approved: boolean` -Whether payment was approved

- gatewayTransactionId: String -Gateway transaction reference
- authorizationCode: String -Authorization code (if approved)
- rawResponse: String -Raw gateway response for audit/debugging

Operations:

- *(Return object; used as output from gateway calls.)*

6.4.16 ReservationService (Design Notes)

Purpose: Handles business logic for reservation creation, seat locking, expiration, and confirmation.

Operations:

- createReservation(userId: UUID, showtimeId: UUID, seatLabels: List<String>): Reservation
- expireReservations(now: Instant): int
- confirmReservation(reservationId: UUID): void

6.4.17 PaymentService

Purpose: Handles payment authorization/charge and updates reservation/payment records accordingly.

Operations:

- authorize(reservationId: UUID, details: PaymentDetails): Transaction
- charge(reservationId: UUID, details: PaymentDetails): Payment

6.4.18 TicketService (Design Notes)

Purpose: Issues tickets (QR payload) after confirmed purchase and sends email confirmation.

Operations:

- issueTickets(reservationId: UUID): List<Ticket>
- emailReceiptAndTickets(userEmail: String, paymentId: UUID): void

6.5.3 Project Timeline (17 Week Schedule)

Weeks 1-3

- Project planning
- Requirements specification development
- Initial repository setup

Weeks 4-6

- Software design specification
- UML class diagram creation
- Software architecture planning

Weeks 7-8

- Database schema implementation
- Initial backend API setup

Weeks 9-11

- Reservation system development
- Seat locking and conflict resolution
- Seat selection user interface integration

Weeks 12-13

- Payment gateway integration
- Transaction logging and retry logic

Weeks 14-15

- Ticket generation system
- QR code payload generation

- Email confirmation service

Weeks 16-17

- Administrative features
- System testing and debugging
- Final documentation and project polish

7. Verification and Validation Test Plan

Github Link:

https://github.com/dnguyensdsu/movieticketingsystem/blob/main/software_requirement_template.pdf

Test Cases:

<https://docs.google.com/spreadsheets/d/1Ov6EEfUtJRcEovYy1mBa07YNTFD4Tp0Z/edit?gid=458716774#gid=458716774>

7.1 Purpose and Scope

This section defines the verification and validation strategy for the Theater Ticketing System. The plan covers payment processing, payment authorization, seat reservation, purchase restrictions, bot prevention, discount processing, administrator management, and the key non-functional requirements already listed in the SRS. The strategy is organized across unit, functional/integration, and system granularities so that isolated logic, multi-component workflows, and full end-to-end behavior are all evaluated.

7.2 Design Specification Update

The Assignment 2 design remains the foundation of the system, but the design model is expanded here to improve test traceability. The updated class view separates policy logic, seat-state logic, security checks, and external service interfaces so that the test plan can point to clear units and interactions.

- The original Payment, Transaction, Reservation, Movie, and Showtime classes remain valid and are retained.
- A new Seat class is added so that seat state transitions (Available -> Locked -> Sold) can be tested directly instead of being implied inside Reservation only.
- A new Ticket class is added to represent QR/barcode generation and confirmation delivery after successful payment.
- DiscountPolicy and PurchasePolicy isolate discount calculation, ticket limit checks, 14-day advance-sale rules, and the post-showtime purchase cutoff into testable units.

- SecurityMonitor, PaymentGateway, EmailService, and CaptchaService are modeled explicitly to improve traceability for bot-prevention, gateway failure, and confirmation-email tests.
- AdminService is shown separately so schedule, pricing, and movie-management features can be validated without mixing them into the customer checkout path.

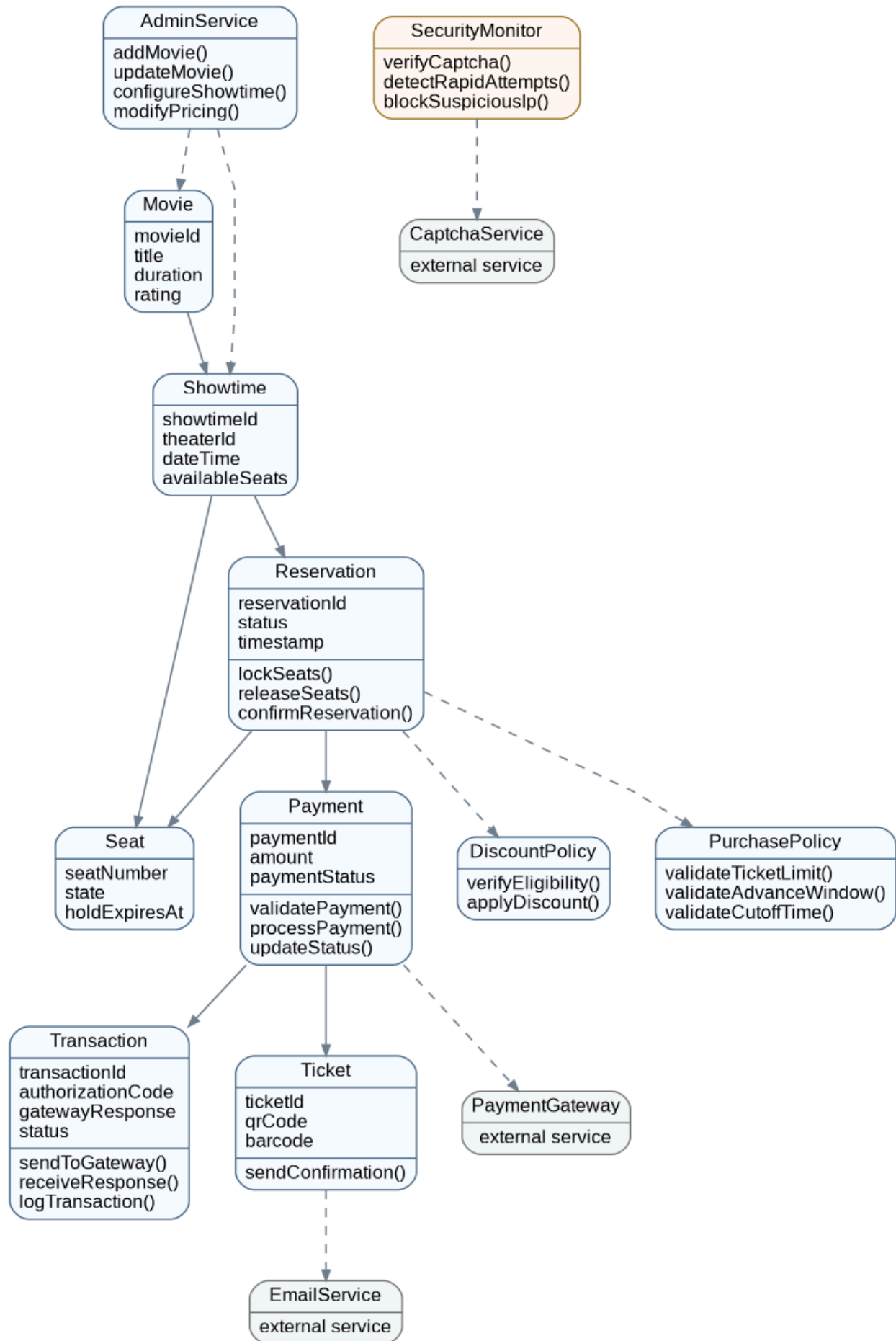


Figure 1. Updated design view used for Assignment 3 testing.

7.3 Test Strategy Overview

Granularity	Test sets	Primary targets	Main failure modes covered
Unit	U-A Payment and policy validation; U-B Reservation state logic	Payment validation, authorization rules, purchase rules, discount rules, seat locking	Missing fields, invalid card format, expired reservations, incorrect discount, rule bypass
Functional / Integration	F-A Checkout integration; F-B Admin update propagation	UI/API/DB/payment-gateway /email interactions and admin changes	Incorrect state handoff, failed retries, stale pricing/showtime data, missing confirmations
System	S-A Peak load and abuse handling; S-B Cross-platform acceptance	Performance, reliability, security, browser/device compatibility	Slow checkout, seat contention, abuse attempts, unavailable flows on supported browsers/devices

7.4 Test Environment and Data

- Test environment: staging deployment with production-like database schema, mock payment gateway, mock email service, and CAPTCHA sandbox.
- Browsers/devices: latest Chrome, Edge, Firefox, and Safari on desktop, plus iOS Safari and Android Chrome.
- Seed data: 3 movies, 6 showtimes, deluxe and regular seating layouts, normal and discounted ticket prices, and multiple user roles (customer, owner/admin, system admin).
- Representative vectors: valid and invalid reservation IDs, seats A10/A11 for available-seat cases, locked seat A12 for contention cases, 21-ticket cart for limit checks, and showtimes inside/outside the 14-day sale window.

7.5 Unit Test Set U-A - Payment and Policy Validation

Purpose: verify isolated business rules before any external integration occurs. These tests focus on methods inside Payment, PurchasePolicy, and DiscountPolicy.

Targeted features: Process Ticket Payment, Payment Authorization Validation, Ticket Purchase Restrictions, and Discount Processing.

Primary vectors: missing CVV, invalid card length, expired card date, 21-ticket cart, showtime more than 14 days away, and eligible student/senior discounts.

Test Case ID	Summary	Coverage
UT-01	Reject incomplete payment input	Missing CVV, blank billing address, or missing reservation ID must return a validation error and must not call the gateway.

UT-02	Reject invalid authorization format	Malformed card numbers or expired cards must fail local validation and log the attempted authorization failure.
UT-04	Apply purchase and discount rules correctly	The policy layer must block more than 20 tickets, block sales beyond the 14-day window, and apply the configured eligible discount exactly once.

7.6 Unit Test Set U-B - Reservation State and Seat Lock Logic

Purpose: verify temporary seat-hold behavior and state transitions without invoking the full checkout flow.

Targeted features: Seat Reservation, reservation timeout handling, and seat-state traceability.

Primary vectors: same seat requested twice, hold timeout at 10 minutes, and successful confirmation after payment.

Test Case ID	Summary	Coverage
UT-03	Lock and release seats safely	Selecting an available seat must move it to Locked, block duplicate holds, and release it automatically when the reservation expires.

7.7 Functional / Integration Test Set F-A - Checkout Integration

Purpose: validate the customer purchase workflow across UI, reservation logic, payment service, transaction logging, and confirmation delivery.

Targeted features: Complete Ticket Payment, Retry Payment, Reservation confirmation, transaction recording, and email/ticket generation.

Primary vectors: approved payment, declined payment followed by retry, valid reservation totals, and successful email dispatch.

Test Case ID	Summary	Coverage
FT-01	Complete approved checkout flow	A valid reservation and approved gateway response must produce a confirmed reservation, stored transaction, generated ticket, and confirmation email.
FT-02	Declined payment then successful retry	A declined payment must keep the reservation pending, display a retry message, and allow a second valid submission to complete successfully before timeout.

7.8 Functional / Integration Test Set F-B - Admin Update Propagation

Purpose: validate that owner/admin changes propagate correctly into customer-facing flows.

Targeted features: Administrator Management, showtime configuration, and pricing updates used by checkout.

Primary vectors: modify ticket price, add a new showtime, and update a seating layout for a deluxe auditorium.

Test Case ID	Summary	Coverage
FT-03	Admin updates are visible to customers	When an admin updates price or showtime data, the browsing and checkout pages must display the new values immediately and compute totals from the new configuration.

7.9 System Test Set S-A - Performance, Concurrency, and Abuse Handling

Purpose: validate end-to-end behavior under realistic theater traffic and hostile usage patterns.

Targeted features: performance, reliability, seat-contention handling, CAPTCHA/abuse controls, and purchase restrictions.

Primary vectors: 1000 concurrent users, simultaneous purchase attempts on the same seat, repeated requests from one IP, 21-ticket carts, and checkout attempts after the showtime cutoff.

Test Case ID	Summary	Coverage
ST-01	Peak-hour performance and concurrency	Under target load, p95 checkout time should remain within 3 seconds, authorization within 2 seconds, and seat reservation conflicts within the allowed threshold.
ST-02	System-wide restriction and anti-bot enforcement	The full deployment must block excessive ticket counts, post-showtime purchases, rapid repeated attempts, and suspicious IP activity while still allowing legitimate users to continue.

7.10 System Test Set S-B - Cross-Platform Acceptance

Purpose: validate the customer experience on supported browsers and mobile devices.

Targeted features: browser compatibility, responsive checkout flow, QR/confirmation delivery, and usability of the seating interface.

Primary vectors: Chrome, Edge, Firefox, Safari, iOS Safari, and Android Chrome using both deluxe and regular theaters.

Test Case ID	Summary	Coverage
ST-03	Cross-browser and mobile acceptance	Browsing, seat selection, checkout, and confirmation must work consistently on all supported browsers and devices without broken layouts or missing steps.

7.11 Requirements Traceability Matrix

Requirement / feature area	Covered by test cases
3.2.1 Process Ticket Payment	UT-01, FT-01, FT-02, ST-01
3.2.2 Payment Authorization Validation	UT-02, FT-01, FT-02
3.2.3 Seat Reservation	UT-03, FT-01, FT-02, ST-01
3.2.4 Ticket Purchase Restrictions	UT-04, ST-02
3.2.5 Bot Prevention	FT-01, ST-02
3.2.6 Discount Processing	UT-04
3.2.7 Administrator Management	FT-03
3.5 Performance / Reliability / Availability	ST-01
3.5 Security / Portability / Usability	ST-02, ST-03

7.12 Exit Criteria

- Unit tests: all P0 unit cases pass and no critical validation defect remains open.
- Functional/integration tests: checkout and admin flows complete without incorrect state propagation, stale data, or missing confirmation artifacts.
- System tests: performance and security thresholds are met, and supported browsers/devices complete the core user journey without blocker defects.